

Mysten Move VM

Security Assessment

July 24th, 2025 — Prepared by OtterSec

Michał Bochnak

embe221ed@osec.io

Sangsoo Kang

sangsoo@osec.io

Dimitri

dshishniashvili@osec.io

Table of Contents

Executive Summary	3
Overview	3
Key Findings	3
Scope	4
Findings	5
Vulnerabilities	6
OS-MVM-ADV-00 Incomplete Detection of Cyclic Dependencies	7
OS-MVM-ADV-01 Stale Jump Table Targets	8
OS-MVM-ADV-02 Incorrect Gas Metering	9
OS-MVM-ADV-03 Improper Type Depth Handling	10
General Findings	11
OS-MVM-SUG-00 Redundant Package Lookup	13
OS-MVM-SUG-01 Missing Version Mapping Update	14
OS-MVM-SUG-02 Unnecessary Resolution Table Updates	15
OS-MVM-SUG-03 Redundant Shared Object Validation	16
OS-MVM-SUG-04 Code Optimization	17
OS-MVM-SUG-05 Unfounded Cache Deadlock Risk	18
OS-MVM-SUG-06 Improving Module Loading Performance	19
OS-MVM-SUG-07 Utilization of Trusted Runtime ID	20
OS-MVM-SUG-08 Out-of-Bounds Panic	21
OS-MVM-SUG-09 Type Mismatch	22
OS-MVM-SUG-10 Misleading Variable Name	23
OS-MVM-SUG-11 Utilization of Storage Package ID	24

OS-MVM-SUG-12 Removal of Redundant Branch Check	25
OS-MVM-SUG-13 Misleading Comment	26
OS-MVM-SUG-14 Improper Error Handling	27
OS-MVM-SUG-15 Typographical Error	28
OS-MVM-SUG-16 Use Typed Serialization	29

Appendices

Vulnerability Rating Scale	30
Procedure	31

01 — Executive Summary

Overview

Mysten engaged OtterSec to assess the `move-vm`. This assessment was conducted between May 22nd and July 15th, 2025. We conducted a follow-up audit of the `sui-adapter` between August 8th and October 22nd, 2025. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 21 findings throughout this audit engagement.

In particular, we identified a vulnerability during cyclic relationship verification, where the program only checks for dependency cycles starting from the first module, potentially missing cycles in disconnected components of the package ([OS-MVM-ADV-00](#)). Additionally, the VM incorrectly undercharges gas by applying cheaper instruction rates, allowing potential abuse of resource-intensive operations ([OS-MVM-ADV-02](#)). Furthermore, the clone implementation fails to increment depth during type substitution, allowing deeply nested types to exceed recursion limits ([OS-MVM-ADV-03](#)).

We also recommended rectifying the incorrect relocation-map type declaration ([OS-MVM-SUG-09](#)), and suggested discarding the check for unconditional branches in the middle of a basic block, as basic blocks may only have branches at the end ([OS-MVM-SUG-12](#)).

We further highlighted refactoring the module-map variable name in the TransactionDataCache structure, as the existing name is misleading ([OS-MVM-SUG-10](#)), and advised improving error handling by returning an invariant violation error rather than silently freeing leftover stack frames after execution, to prevent masking of potential VM logic bugs ([OS-MVM-SUG-14](#)).

Moreover, multiple instances of duplicated or unnecessary code may be removed, including unnecessary resolution table updates ([OS-MVM-SUG-02](#)), redundant shared object validation logic ([OS-MVM-SUG-03](#)), irrelevant logging ([??](#)), and needless package reloads from the data store when they already exist in the runtime ([OS-MVM-SUG-04](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/MystenLabs/sui>. This audit was performed against [57c5091](#). The follow-up audit was performed on commits from [023c3dc](#) to [953d2ba](#).

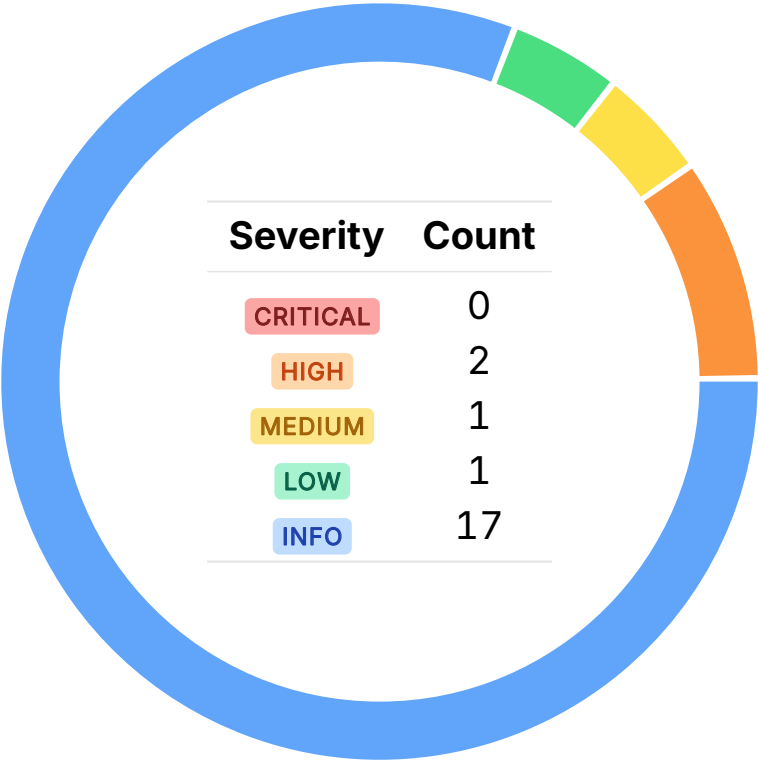
A brief description of the program is as follows:

Name	Description
move-vm	A major Move VM rewrite improving performance, modularity, and memory efficiency through per-package caching, arena-based loading, and a refactored interpreter. It also includes the separation of runtime and execution layers.
sui-adapter	The new Sui Adapter statically loads and verifies programmable transactions, performing all type, reference, memory, and drop safety checks before execution. It enforces strict invariants, precomputes package linkage, and uses a borrow-graph model to prevent use-after-move or dangling references.

03 — Findings

Overall, we reported 21 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-MVM-ADV-00	HIGH	RESOLVED ✓	During cyclic relationship verification, the program only checks for dependency cycles starting from the first module, potentially missing cycles in disconnected components of the package.
OS-MVM-ADV-01	HIGH	RESOLVED ✓	Failure to update <code>VariantJumpTable</code> after block renumbering leads to stale jump targets.
OS-MVM-ADV-02	MEDIUM	RESOLVED ✓	The VM incorrectly undercharges gas for <code>MutBorrowFieldGeneric</code> , <code>CastU32</code> , and <code>CastU256</code> by applying cheaper instruction rates, allowing potential abuse of resource-intensive operations.
OS-MVM-ADV-03	LOW	RESOLVED ✓	<code>clone_impl</code> fails to increment <code>depth</code> during type substitution, allowing deeply nested types to exceed recursion limits.

Incomplete Detection of Cyclic Dependencies HIGH

OS-MVM-ADV-00

Description

In the current implementation of `package_resolution`, while verifying cyclic relationships in packages `verify_package_no_cyclic_relationships` does not accurately detect dependency cycles. The function incorrectly assumes that the module graph of a package is fully connected and only checks for cycles starting from the first module (`cyclic_dependencies::verify_module` runs DFS only from the initial module). As a result, if the package contains disconnected components, any cycles within those components are never detected. This creates a vulnerability where cyclic dependencies can silently pass verification. Such cycles violate Move's module dependency rules.

```
>_ move-vm-runtime/src/runtime/package_resolution.rs
```

RUST

```
fn verify_package_no_cyclic_relationships(
    package: &[&Module],
    cached_packages: &BTreeMap<PackageStorageId, &Package>,
    relocation_map: &HashMap<PackageStorageId, RuntimePackageId>,
) -> VMResult<()> {
    [...]

    cyclic_dependencies::verify_module(&module.value, |runtime_module_id| {
        let module = if let Some(bundled) = bundle_verified.get(runtime_module_id) {
            Some(**bundled)
        }[...]
        module
            .map(|m| m.value.immediate_dependencies())
            .ok_or_else(|| PartialVMError::new(StatusCode::MISSING_DEPENDENCY))
    })?;
    Ok(())
}
```

Remediation

Update the verification logic to iterate through all modules to ensure complete cycle detection.

Patch

Resolved in [a20fda5](#).

Stale Jump Table Targets HIGH

OS-MVM-ADV-01

Description

During JIT translation, `flatten_and_renumber_blocks` concatenates the basic blocks and updates branch targets to match the new block layout, but fails to update the `VariantJumpTable`. This results in stale jump targets, creating incorrect or undefined control flow.

Remediation

Ensure that all jump-related metadata, including the `VariantJumpTable`, is updated consistently.

Patch

Resolved in [a8611c7](#).

Incorrect Gas Metering MEDIUM

OS-MVM-ADV-02

Description

`eval::op_step_impl` contains gas metering bugs where incorrect `SimpleInstruction` variants are utilized for certain bytecodes. Specifically, `MutBorrowFieldGeneric` is always charged as `ImmBorrowFieldGeneric` due to a faulty pattern match, and both `CastU32` and `CastU256` incorrectly charge the cheaper `CastU16` rate. These undercharges break the VM's cost model, allowing the possibility of performing more expensive operations at reduced gas cost, potentially enabling unfair execution.

Remediation

Refactor the logic to ensure the appropriate gas is charged for the above operations.

Patch

Resolved in [b80c30b](#) and [dee560a](#).

Improper Type Depth Handling LOW

OS-MVM-ADV-03

Description

`apply_subst` in `clone_impl` performs type cloning utilizing a substitution closure that ignores the depth argument. This prevents proper tracking of recursive nesting depth during type substitution. As a result, deeply nested types such as `Vector<Vector<...>>` may bypass the `TYPE_DEPTH_MAX` limit. For example, `Vector<u64>` should reach `depth = 2`, while `Vector<T>` substitution with `u64` will reach `depth = 1`, thus, failing to reflect the increasing depth.

```
>_ move-vm-runtime/src/jit/execution/ast.rs
```

RUST

```
fn clone_impl(&self, depth: usize) -> PartialVMResult<Type> {  
    self.apply_subst(|idx, _| Ok(Type::TyParam(idx)), depth)  
}
```

Remediation

Pass the `depth` incremented by 1 to `apply_subst`.

Patch

Resolved in [5f30365](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MVM-SUG-00	<code>publish_and_init_package</code> and <code>upgrade</code> redundantly reloads dependency packages from on-chain state via the legacy <code>fetch_packages</code> implementation, even though these dependencies were already resolved and cached in memory.
OS-MVM-SUG-01	<code>new_for_publication</code> does not update the <code>versions</code> field in <code>ResolvedLinkage</code> , leaving the new package's version undefined.
OS-MVM-SUG-02	Duplicate updates to <code>all_versions_resolution_table</code> and a <code>get_package</code> call should be removed, as <code>add_and_unify</code> already handles these internally.
OS-MVM-SUG-03	The shared object validation in <code>context::finish</code> here is redundant and may be removed.
OS-MVM-SUG-04	Redundant validation of already cached packages in <code>resolve_packages</code> may be avoided to improve performance.
OS-MVM-SUG-05	There is no possibility of deadlocks on <code>MoveCache::cache</code> as it accesses the cache through <code>cached_package_at</code> and <code>add_to_cache</code> , which each acquire and release a single lock independently.
OS-MVM-SUG-06	The module loading loop in <code>translate::package</code> is inefficient, resulting in a $O(n^2)$ complexity.

OS-MVM-SUG-07	<code>translate::package</code> re-derives <code>runtime_id</code> from the first module instead of utilizing the trusted <code>runtime_id</code> already in <code>SerializedPackag</code> .
OS-MVM-SUG-08	The bounds check in <code>eval::step</code> is off-by-one, allowing <code>pc</code> to equal the array length and resulting in out-of-bounds panics when accessing <code>instructions[pc]</code> .
OS-MVM-SUG-09	The <code>relocation_map</code> type declaration in <code>linkage</code> is Incorrect.
OS-MVM-SUG-10	The <code>module_map</code> variable name in the <code>data_cache::TransactionDataCache</code> structure is misleading and should be updated.
OS-MVM-SUG-11	<code>validate_for_publish</code> and <code>validate_for_vm_executionuse</code> utilize <code>RuntimePackageId</code> instead of <code>PackageStorageId</code> .
OS-MVM-SUG-12	The check for unconditional branches in the middle of a basic block is unnecessary, as basic blocks may only have branches at the end.
OS-MVM-SUG-13	The comment describing the <code>defining_ids</code> field in <code>ast::PackageVirtualTable</code> and <code>ast::VMDispatchTables</code> is misleading.
OS-MVM-SUG-14	<code>eval::run</code> currently hides potential VM logic bugs by silently freeing leftover stack frames after execution.
OS-MVM-SUG-15	There is a typographical error in the error message in <code>dispatch_tables::load_type</code> .
OS-MVM-SUG-16	Using typed serialize is recommended over raw serialization.

Redundant Package Lookup

OS-MVM-SUG-00

Description

In `publish_and_init_package` and `upgrade`, dependencies are fetched via `fetch_packages` from the `linkable_store`, even though they were already resolved during transaction loading.

```
>_ sui-adapter/src/static_programmable_transactions/execution/context.rs RUST

pub fn publish_and_init_package<Mode: ExecutionMode>([...]) -> Result<ObjectID, ExecutionError>
    ↪ {
    [...]
    let dependencies = self.fetch_packages(dep_ids)?;
    let package = Rc::new(MovePackage::new_initial(
        &modules,
        self.env.protocol_config,
        dependencies.iter().map(|p| p.move_package()),
    ));
    let package_id = package.id();
    [...]
}
```

`fetch_packages` calls `legacy_ptb::execution::fetch_packages` (as shown below) to reload dependency packages from on-chain state. This results in inefficiency due to the redundant lookup. Instead, dependencies should be retrieved directly from `cached_package_store` to leverage cached `MovePackage` instances.

```
>_ sui-adapter/src/static_programmable_transactions/execution/context.rs RUST

fn fetch_packages(
    &mut self,
    dependency_ids: &[ObjectID],
) -> Result<Vec<PackageObject>, ExecutionError> {
    legacy_ptb::execution::fetch_packages(&self.env.state_view, dependency_ids)
}
```

Remediation

Utilize `cached_package_store` for improved efficiency, as it holds all previously loaded `MovePackages` in memory.

Patch

Resolved in [12c703c](#).

Missing Version Mapping Update

OS-MVM-SUG-01

Description

`resolved_linkage::new_for_publication` updates ID mappings but does not update the `ResolvedLinkage.versions` field, leaving the new package's version undefined, resulting in incomplete linkage metadata.

```
>_ sui-adapter/src/static_programmable_transactions/linkage/resolved_linkage.rs
```

RUST

```
pub fn new_for_publication(
    link_context: ObjectID,
    original_package_id: ObjectID,
    mut resolved_linkage: ResolvedLinkage,
) -> RootedLinkage {
    // original package ID maps to the link context (new package ID) in this context
    resolved_linkage
        .linkage
        .insert(original_package_id, link_context);
    // Add resolution from the new package ID to the original package ID.
    resolved_linkage
        .linkage_resolution
        .insert(link_context, original_package_id);
    let resolved_linkage = Rc::new(resolved_linkage);
    Self {
        link_context: *link_context,
        resolved_linkage,
    }
}
```

Remediation

Ensure to update the `ResolvedLinkage.versions` field in `new_for_publication`.

Patch

Resolved in [13bf451](#) by removing `ResolvedLinkage.versions`.

Unnecessary Resolution Table Updates

OS-MVM-SUG-02

Description

`resolution_table_with_native_packages` and `compute_publication_linkage` (shown below) redundantly update `all_versions_resolution_table`, even though they both call `add_and_unify`, which already performs this update internally. Similarly, there is no requirement to explicitly call `get_package` in `compute_publication_linkage`, as `add_and_unify` already fetches and processes the package.

```
>_ sui-adapter/src/static_programmable_transactions/linkage/legacy_linkage.rs RUST

// Compute the linkage for a publish or upgrade command. This is a special case because
pub(crate) fn compute_publication_linkage(
    &self,
    deps: &[ObjectID],
    store: &dyn PackageStore,
) -> Result<ResolutionTable, ExecutionError> {
    for id in deps {
        let pkg = get_package(id, store)?;
        add_and_unify(id, store, &mut resolution_table, ConflictResolution::exact)?;
        resolution_table
            .all_versions_resolution_table
            .insert(pkg.id(), pkg.original_package_id());
    }
    Ok(resolution_table)
}
```

Remediation

Remove the updates to `all_versions_resolution_table` in the mentioned functions.

Patch

Resolved in [6ff2075](#).

Redundant Shared Object Validation

OS-MVM-SUG-03

Description

`static_ptb::finish` calls `legacy_ptb::finish` at the end of its execution, rendering the shared object validation loop in `static_ptb::finish` redundant since `legacy_ptb::finish` already enforces the same checks. Remove this loop to simplify the code and eliminate duplicate execution.

Remediation

Remove the redundant code.

Patch

Resolved in [3b825ac](#).

Code Optimization

OS-MVM-SUG-04

Description

`resolve_packages` redundantly validates packages already present in the cache, even though cached entries have already been verified and JIT-compiled. This results in unnecessary overhead during repeated package lookups.

```
>_ move -vm-runtime/src/runtime/package_resolution.rs
```

RUST

```
pub fn load_and_verify_packages(
    vm_config: &VMConfig,
    natives: &NativeFunctions,
    data_store: &impl DataStore,
    allow_loading_failure: bool,
    packages_to_read: &BTreeSet<PackageStorageId>,
) -> VMResult<Vec<verification::ast::Package>> {
    let packages = packages_to_read.iter().cloned().collect::<Vec<_>>();
    let packages = match data_store.load_packages(&packages) {
        Ok(packages) => packages,
        Err(err) if allow_loading_failure => return Err(err),
        Err(err) => { error!("[VM] Error fetching packages {packages_to_read:?}");
            return Err(expect_no_verification_errors(err));
        }
    };
    [...]
```

Remediation

Do not reload packages from the data store if they already exist in the runtime cache to improve performance.

Unfounded Cache Deadlock Risk

OS-MVM-SUG-05

Description

Since `MoveCache` utilizes a single lock for all operations and all access goes through controlled methods (`cached_package_at` and `add_to_cache`) with the possibility of locking only a single in `MoveRuntime`, there is no risk of deadlock. Also, in `package_resolution::jit_package_for_publish`, the cache is accessed twice via `cached_package_at`, acquiring the read lock twice unnecessarily. This is inefficient compared to `resolve_packages`, where the cache is accessed only once via `if let Some(...)`.

```
>_ move-vm-runtime/src/runtime/package_resolution.rs
```

RUST

```
pub fn jit_package_for_publish([...]) -> VMResult<Arc<move_cache::Package>> {  
    let storage_id = verified_pkg.storage_id;  
    if cache.cached_package_at(storage_id).is_some() {  
        return Ok(cache.cached_package_at(storage_id).unwrap());  
    }  
    [...]  
}
```

Remediation

Utilize `cached_package_at` once in `jit_package_for_publish`.

Patch

Resolved in [PR-24396](#).

Improving Module Loading Performance

OS-MVM-SUG-06

Description

The current module loading loop in `translate::package` has $O(n^2)$ complexity due to repeated dependency checks and re-insertions. Replace it with a proper topological sort to improve performance by resolving module dependencies in a single pass. Replace it with a proper topological sort to enhance performance by resolving module dependencies in a single pass.

Remediation

Update `translate::package` for improved time complexity.

Utilization of Trusted Runtime ID

OS-MVM-SUG-07

Description

The current code in `translate::package` re-derives the `runtime_id` from the first module in the package, even though the `SerializedPackage` already includes a trusted `runtime_id` from the data store. Instead, the function should utilize the provided `runtime_id` and validate that all modules match it.

```
>_ move-vm-runtime/src/validation/deserialization/translate.rs
```

RUST

```
pub(crate) fn package(vm_config: &VMConfig, pkg: SerializedPackage) -> VMResult<Package> {  
    [...]  
    let runtime_id = *modules.keys().next().expect("non-empty package").address();  
    Ok(Package::new(  
        runtime_id,  
        modules.into_values().collect(),  
        pkg,  
    ))  
}
```

Remediation

Ensure `translate::package` utilizes the provided `runtime_id` and validates that all modules match it.

Patch

Resolved in [PR-24398](#).

Out-of-Bounds Panic

OS-MVM-SUG-08

Description

The existing `pc <= instructions.len` check in `eval::step` is incorrect because accessing `instructions[pc]` when `pc == len` results in an out-of-bounds panic.

```
>_ move-vm-runtime/src/execution/interpreter/eval.rs
```

RUST

```
fn step(
    state: &mut MachineState,
    run_context: &mut RunContext,
    gas_meter: &mut impl GasMeter,
) -> VMResult<StepStatus> {
    let fun_ref = state.call_stack.current_frame.function();
    let instructions = fun_ref.code();
    let pc = state.call_stack.current_frame.pc as usize;
    assert!(
        pc <= instructions.len(),
        "PC beyond instruction count for {}",
        fun_ref.name()
    );
    [...]
}
```

Remediation

Replace the check with `pc < instructions.len` to ensure the program counter always refers to a valid instruction.

Patch

Resolved in [PR-24229](#).

Type Mismatch

OS-MVM-SUG-09

Description

`verify_package_valid_linkage` and `verify_package_no_cyclic_relationships` (shown below) in `linkage` declare `relocation_map` as `HashMap<PackageStorageId, RuntimePackageId>`, while the actual type should be `HashMap<RuntimePackageId, PackageStorageId>`, as utilized in `verify_linkage_and_cyclic_checks_for_publication`. While this type mismatch does not create any immediate issues, it is misleading and should be rectified for consistency and to improve code clarity.

```
>_ move-vm-runtime/src/validation/verification/linkage.rs RUST

fn verify_package_no_cyclic_relationships(
    package: &[&Module],
    cached_packages: &BTreeMap<PackageStorageId, &Package>,
    relocation_map: &HashMap<PackageStorageId, RuntimePackageId>,
) -> VMResult<> { [...]
```

Remediation

Rectify the type mismatch.

Patch

Resolved in [PR-24400](#) and [PR-23841](#).

Misleading Variable Name

OS-MVM-SUG-10

Description

The `module_map` variable name in the `data_cache::TransactionDataCache` structure is misleading, as it maps account addresses to full packages, not individual modules. Renaming it to `package_map` will better reflect its purpose and improve code clarity.

```
>_ move -vm-runtime/src/runtime/data_cache.rs
```

RUST

```
pub struct TransactionDataCache<S> {  
    pub remote: S,  
    module_map: BTreeMap<AccountAddress, AccountDataCache>,  
}
```

Remediation

Rename the `module_map` variable as stated above.

Patch

Resolved in [PR-23729](#).

Utilization of Storage Package ID

OS-MVM-SUG-11

Description

Both `validate_for_publish` and `validate_for_vm_execution` in `validation` utilize `RuntimePackageId`. However, `PackageStorageId` will be more appropriate, as they operate on logical package identities for structural validation rather than runtime-specific instances.

```
>_ move-vm-runtime/src/validation/mod.rs
```

RUST

```
pub fn validate_for_publish(
    natives: &NativeFunctions,
    vm_config: &VMConfig,
    runtime_package_id: RuntimePackageId,
    package: SerializedPackage,
    dependencies: BTreeMap<RuntimePackageId, &verification::ast::Package>,
) -> VMResult<verification::ast::Package> {
    [...]
}

pub fn validate_for_vm_execution(
    packages: BTreeMap<RuntimePackageId, &verification::ast::Package>,
) -> VMResult<()> {
    // TODO: Nothing else to do, right?
    verify_linkage_and_cyclic_checks(&packages)
}
```

Remediation

Update `validate_for_publish` and `validate_for_vm_execution` to utilize `PackageStorageId`.

Patch

Resolved in [PR-24401](#).

Removal of Redundant Branch Check

OS-MVM-SUG-12

Description

The code in `dead_code_elim::eliminate_unreachable` that clears all prior instructions after an unconditional branch is unnecessary because, in Move, such branches always end a basic block.

```
>_ move-vm-runtime/src/jit/optimization/optimizations/dead_code_elim.rs RUST

fn eliminate_unreachable(context: &mut BlockContext<'_, '_, '_>, code: &mut Vec<ast::Bytecode>)
    → {
    use ast::Bytecode;
    let mut output_code = vec![];
    while let Some(instr) = code.pop() {
        [...]
        if instr.is_unconditional_branch() {
            *context.changed = true;
            output_code = vec![];
        }
        output_code.push(instr);
    }[...]
}
```

Remediation

Remove the unconditional branch check.

Patch

Resolved in [PR-24428](#).

Misleading Comment

OS-MVM-SUG-13

Description

The comment describing the `defining_ids` field in `ast::PackageVirtualTable` and `ast::VMDispatchTables` (shown below) utilizes the word “mentioned”, which is misleading, as it implies inclusion of referenced types. In reality, the field only includes package IDs for types that are defined within the package.

```
>_ move-vm-runtime/src/jit/execution/ast.rs
```

RUST

```
#[derive(Debug)]
pub struct VMDispatchTables {
    [...]
    /// Defining ID Set -- a set of all defining IDs on any types mentioned in the package.
    /// [SAFETY] Ordering is not guaranteed
    #[allow(dead_code)]
    pub(crate) defining_id_origins: BTreeMap<PackageStorageId, RuntimePackageId>,
}
```

Remediation

Replace “mentioned” with “defined” for clarity and to prevent ambiguity regarding the purpose of the field.

Patch

Resolved in [PR-24402](#).

Improper Error Handling

OS-MVM-SUG-14

Description

`eval::run` currently frees any leftover call stack frames after execution completes, potentially masking bugs. Since the VM is expected to empty the call stack during execution, leftover frames indicate a logic error.

```
>_ move-vm-runtime/src/execution/interpreter/eval.rs
```

RUST

```
pub(super) fn run(
    start_state: MachineState,
    vtables: &mut VMDispatchTables,
    vm_config: Arc<VMConfig>,
    extensions: &mut NativeContextExtensions,
    tracer: &mut Option<VMTracer<'_>>,
    gas_meter: &mut impl GasMeter,
) -> VMResult<Vec<Value>> {
    [...]
    heap.free_stack_frame(current_frame.stack_frame)
        .map_err(|e| e.finish(Location::Undefined))?;
    for frame in frames.into_iter().rev() {
        heap.free_stack_frame(frame.stack_frame)
            .map_err(|e| e.finish(Location::Undefined))?;
    }
    Ok(operand_stack.value)
}
```

Remediation

Ensure the function returns an invariant violation error to enforce execution correctness.

Patch

Resolved in [PR-24424](#).

Typographical Error

OS-MVM-SUG-15

Description

There is a typographical error in the error message in `dispatch_tables::load_type`, where there is a mismatch in the defining ID. The same placeholder `defining_id` is utilized twice. The expected ID is from `defining_id`, and the actual is from `datatype.defining_id.address`.

```
>_ move-vm-runtime/src/execution/dispatch_tables.rs RUST

pub(crate) fn load_type(&self, type_tag: &TypeTag) -> VMResult<Type> {
    Ok(match type_tag {
        [...]
        TypeTag::Struct(struct_tag) => {
            [...]
            // The defining ID should match the defining ID of the datatype that we
            // have loaded.
            if datatype.defining_id.address() != &defining_id {
                return Err(
                    PartialVMError::new(StatusCode::UNKNOWN_INVARIANT_VIOLATION_ERROR)
                        .with_message(format!(
                            "Defining ID {defining_id} does not match defining ID {defining_id}"
                        ))
                        .finish(Location::Undefined),
                );
            }
        }
    })
}
```

Remediation

Utilize `datatype.defining_id.address` and `defining_id` as the placeholders to correctly show the expected and actual values when a mismatch occurs.

Patch

Resolved in [PR-23827](#).

Use Typed Serialization

OS-MVM-SUG-16

Description

In this [PR](#), `serialize` for serialization without a layout has been introduced, and the previous method has been renamed to `typed_serialize`. However, since raw serialization without a layout may not guarantee type safety if proper type checking isn't performed, it is recommended to use `typed_serialize` to ensure type safety.

```
> _ move-vm-types/src/values/values_impl.rs
```

RUST

```
pub fn typed_serialize(&self, layout: &MoveTypeLayout) -> Option<Vec<u8>> {
    bcs::to_bytes(&AnnotatedValue {
        layout,
        val: &self.0,
    })
    .ok()
}

pub fn serialize(&self) -> Option<Vec<u8>> {
    bcs::to_bytes(&self.0).ok()
}
```

Remediation

Do not use the raw serialization with `bcs::to_bytes`.

Patch

Resolved in [PR-24311](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.